

Akran Uygulaması — Grup Görevi

EventHub — Etkinlik Takip Uygulaması

Süre: 1 hafta

Genel Bakış

Bu görevde 3 kişilik gruplar halinde React Native kullanarak çok ekranlı bir mobil uygulama geliştireceksiniz. Uygulama DummyJSON API'sini kullanacak ve ekranlar arasında gerçek bir veri akışı olacak.

Görevin temel amacı ekran saymak değil; birden fazla ekranın aynı veriyi farklı şekillerde gösterdiği, kullanıcı eylemlerinin anlık olarak tüm uygulamayı etkilediği bir mimari kurmaktır.

API: DummyJSON

Tüm veriler DummyJSON API'sinden gelecektir. Kayıt gerektirmez, tamamen ücretsizdir.

Base URL

<https://dummyjson.com>

Kullanacağınız endpointler

Endpoint	Ne İçin	Not
POST /auth/login	Giriş (JWT token)	username: emilys / password: emilyspass
GET /auth/me	Token ile kullanıcı bilgisi	Header: Authorization: Bearer {token}
GET /products?limit=20	Etkinlik listesi	title, description, thumbnail, stock → kontenjan
GET /products/{id}	Etkinlik detayı	Tek ürün/etkinlik
GET /products/search?q=...	Arama	Feed ekranında kullan
GET /products/categories	Kategoriler	Filtre için kullanabilirsin

Zorunlu: Context API Mimarisi

Bu görevde useState tek bir ekranın içinde kullanılabilir. Ekranlar arasında paylaşılan her veri bir Context üzerinden yönetilmek zorundadır.

Kurmanız gereken üç Context:

Context	İçerdiği State	Kim Kullanıyor
AuthContext	user, token, login(), logout()	Login → tüm ekranlar → Settings
EventContext	events, favorites, registrations, addFavorite(), register(), updateStock()	Feed ↔ Detail ↔ Favorites ↔ Profile
ThemeContext	theme ('light'/'dark'), toggleTheme()	Settings → tüm ekranlar

⚠ Context yapısını kurmadan ekran geliştirmeye başlamayın. Önce Context'leri tanımlayın, sonra ekranlara bölün. Aksi halde bağlantıları sonradan kurmak çok daha zor olacak.

Ekranlar ve Gereksinimler

6 ekran 3 kişiye ikiye bölünür. Her kişi kendi ekranlarını geliştirir, ancak Context üzerinden diğer ekranlarla gerçek zamanlı etkileşim sağlamak zorundadır.

Ekran 1 — Login

Kişi	Kişi A
Endpoint	POST /auth/login
Zorunlu alanlar	E-posta alanı (DummyJSON için username kabul eder), şifre alanı, giriş butonu
Validasyon	Her iki alan boşsa hata mesajı göster. Şifre alanı maskelenmiş olmalı (secureTextEntry).
Başarılı giriş	Token ve kullanıcı bilgisi AuthContext'e kaydedilmeli. Uygulama Feed ekranına yönlendirilmeli.
Başarısız giriş	API'den dönen hata mesajı kullanıcıya gösterilmeli. Ekran temizlenmemeli.
Loading state	İstek süresince buton devre dışı, loading göstergesi aktif olmalı.
Test kullanıcısı	username: emilys password: emilyspass

⚠ Login ekranı zaten oturum açıksa (token AuthContext'te varsa) gösterilmemeli. Doğrudan Feed'e yönlendirmeli.

Ekran 2 — Feed (Etkinlik Listesi)

Kişi	Kişi B
Endpoint	GET /products?limit=20 GET /products/search?q=...
Liste	FlatList kullan. Her kart: etkinlik görseli, başlık, kategori, kontenjan sayacı.
Kontenjan durumu	stock > 0 ise normal görün. stock === 0 ise kart grileşmeli ve 'Kontenjan doldu' etiketi çıkmalı.
Arama	Üstte arama çubuğu. Kullanıcı yazdıkça /products/search endpointi tetiklenmeli (debounce uygula).
Favori ikonu	Her kartta favori ikonu. EventContext üzerinden addFavorite() çağrılmalı. İkon anlık güncellenmeli.
Tıklama	Karta tıklayınca Detail ekranına git. Seçilen etkinliği parametre olarak geç.
Pull-to-refresh	Aşağı çekerek listeyi yenile.

⚠ Feed ekranı EventContext'teki favorites state'ini dinlemeli. Kullanıcı Favorites ekranından bir etkinliği kaldırırsa Feed'deki ikon da güncellenmeli.

Ekran 3 — Detail (Etkinlik Detayı)

Kişi	Kişi C
Veri kaynağı	Feed'den navigation parametresi olarak gelen etkinlik objesi
Gösterilecekler	Büyük görsel, başlık, açıklama, kategori, mevcut kontenjan, fiyat (bilet ücreti gibi göster).
Kayıt Ol butonu	Kontenjan > 0 ise aktif. Tıklayınca EventContext'teki updateStock() ve register() çağrılmalı. Kontenjan 1 azalmalı.
Anlık güncelleme	Kayıt sonrası buton 'Kayıt İptal Et' haline gelmeli. Kontenjan sayacı ekranda anlık düşmeli.
Favori butonu	Favori durumu EventContext'ten okunmalı. Toggle yapılabilmesi.
Kontenjan sıfır	Buton 'Kontenjan Doldu' yazmalı ve devre dışı olmalı.
Geri dön	Feed'e dönerken Feed'deki kart güncellenmeli (Context sayesinde otomatik).

⚠ updateStock() işlemi EventContext üzerinden yapılmalı. Detail ekranı kendi local state'inde stok tutmamalı. Bu sayede Feed ekranı da aynı anda güncellenir.

Ekran 4 — Favorites (Favoriler)

Kişi	Kişi B
Veri kaynağı	EventContext'teki favorites dizisi
Liste	Favori etkinlikler FlatList ile gösterilmeli. Boşsa 'Henüz favori eklemediniz' mesajı.
Kaldırma	Her öğede 'Kaldır' butonu. EventContext'teki removeFavorite() çağrılmalı.
Detail'e git	Favori öğeye tıklayınca Detail ekranına gidin.
Anlık senkron	Feed'de favori eklenen bir etkinlik bu ekranda anında görünmeli. Sayfa yenilemesi gerekmemeli.
Kayıt durumu	Favorideki etkinlik aynı zamanda 'kayıtlı' ise bunu belirt (ikon veya etiket).

⚠ Bu ekran tamamen EventContext'e bağımlıdır. Kendi API çağrısı yapmamalı. Context değişince otomatik yenilenmeli.

Ekran 5 — Profile (Profil)

Kişi	Kişi C
Kullanıcı verisi	AuthContext'ten al. GET /auth/me endpointi ile tamamla (ad, soyad, e-posta, profil fotoğrafı).
İstatistikler	EventContext'ten hesapla: kaç etkinliğe kayıt olundu, kaç favori var.
Kayıtlı etkinlikler	registrations dizisini listele. Her birinin adı ve kontenjan durumu görünmeli.
Kayıt iptali	Listeden bir etkinliği iptal edebilmeli. EventContext'teki unregister() ve updateStock() (+1) çağrılmalı.
İptal sonrası	Detail ekranına geri gidince buton tekrar 'Kayıt Ol' haline gelmeli.
Tema	ThemeContext'ten gelen tema ile arka plan ve yazı renkleri değişmeli.

⚠ İstatistik sayıları (kayıt sayısı, favori sayısı) EventContext her değiştiğinde otomatik güncellenmeli. Ayı bir state tutmaya gerek yok.

Ekran 6 — Settings (Ayarlar)

Kişi	Kişi A
Tema	Dark / Light toggle. ThemeContext'teki toggleTheme() çağrılmalı. Tüm uygulama anlık değişmeli.
Bildirimler	Toggle switch (dummy olabilir, state tutmak yeterli).
Dil seçimi	Picker/dropdown (dummy olabilir — TR / EN).
Hesap bilgisi	AuthContext'ten kullanıcı adı ve e-posta göster.
Çıkış Yap	AuthContext'teki logout() çağrılmalı. Token ve kullanıcı bilgisi temizlenmeli. Login ekranına yönlendirmeli.
Tema tutarlılığı	Settings ekranı da ThemeContext'i dinlemeli. Kendi teması da toggle'a göre değişmeli.

⚠ Çıkış Yap butonuna basınca EventContext de sıfırlanmalı mı? Bu bir tasarım kararı — grubunuz karar vermeli ve nedenini açıklayabilmeli.

Ekranlar Arası Veri Akışı

Aşağıdaki senaryolar doğru çalışmalıdır. Bunlar değerlendirmede test edilecek:

Kullanıcı ne yapar?	Ne değişmeli?	Hangi ekranlar etkilenir?
Detail'de 'Kayıt Ol' tıklar	Kontenjan 1 düşer, buton değişir, profildeki sayaç artar	Detail + Feed + Profile
Feed'de favori ikonuna tıklar	İkon dolar, Favorites'e etkinlik eklenir	Feed + Favorites
Favorites'ten etkinlik kaldırır	Feed'deki ikon boşalır, Favorites listesi kısalır	Favorites + Feed
Profile'dan kayıt iptal eder	Kontenjan +1 artar, Detail'de buton 'Kayıt Ol' olur	Profile + Detail + Feed
Settings'ten Dark Mode açar	Tüm ekranların arka planı ve yazı renkleri değişir	Tüm uygulama
Settings'ten çıkış yapar	Token temizlenir, Login'e yönlendirilir, stack sıfırlanır	Tüm uygulama

Aşama 1 — Figma Tasarımı (Koddan Önce)

Koda geçmeden önce tüm ekranlar Figma'da tasarlanmalıdır. Tasarım tamamlanmadan geliştirme başlayamaz.

Figma'da yapılması gerekenler

- Her 6 ekran için tam ekran (375×812 px — iPhone 14 frame) tasarım

- Tutarlı bir renk paleti ve tipografi sistemi (Figma'da Styles olarak tanımlanmalı)
- Reusable component'lar Figma'da da component olarak tanımlanmalı (etkinlik kartı, buton, input vb.)
- Dark mode için ayrı frame'ler (en az Feed ve Settings ekranı için)
- Tüm interaktif durumlar gösterilmeli: boş liste, loading, hata, kontenjan doldu, favori aktif/pasif

⚠ Figma linki teslimata dahil edilecektir. 'View only' erişim açık olmalı. Tasarım eksik olan grupların kodu değerlendirilmez.

Figma → Kod kuralı

Uygulamanın görünümü Figma tasarımıyla birebir örtüşmelidir. Sunumda Figma ve uygulama yan yana karşılaştırılacaktır. Renk, boşluk, tipografi, ikon ve layout farklılıkları puan kırar.

Zorunlu: Component Yapısı

Her şeyi ekran dosyasının içine yazmak yasaktır. Aşağıdaki kurallara uyulmalıdır:

Klasör	Ne gider?
/screens	Ekran dosyaları — sadece layout ve Context bağlantısı. İş mantığı burada olmaz.
/components	Tekrar kullanılabilir UI parçaları: EventCard, Button, Avatar, Badge, EmptyState vb.
/context	AuthContext.js, EventContext.js, ThemeContext.js — her biri ayrı dosyada
/hooks	Özel hook'lar: useAuth(), useEvents(), useTheme()
/services	API çağrıları: api.js veya ayrı servis dosyaları (authService, eventService)
/constants	Renkler, font boyutları, spacing değerleri — magic number yasak

⚠ Figma'da component olarak tanımladığınız her şeyin kodda da ayrı bir component dosyası olması bekleniyor. Tasarım ile kod yapısı paralel gitmelidir.

GitHub Kuralları

Proje tek bir ortak repoda geliştirilecektir. Aşağıdaki kurallar zorunludur:

Repo yapısı

- Repo grubun ilk toplantısında oluşturulacak ve herkese erişim verilecek
- main branch'i korunacak — doğrudan push yasak
- Her kişi kendi branch'inde çalışacak: kisi-a/login-settings, kisi-b/feed-favorites, kisi-c/detail-profile

- Değişiklikler Pull Request ile main'e alınacak — en az 1 kişinin review'u zorunlu

Commit kuralları

- Her anlamlı değişiklik ayrı commit olmalı — 'tüm projeyi yaptım' tek commiti kabul edilmez
- Commit mesajları Türkçe veya İngilizce, açıklayıcı olmalı
- Örnek geçerli commitler:
 - feat: Login ekranı validasyon eklendi
 - feat: EventContext registerEvent fonksiyonu
 - fix: Feed kontenjan sıfırsa buton disabled
 - style: EventCard dark mode renkleri düzenlendi
- Örnek geçersiz commitler:
 - 'bitti', 'değişiklikler', 'asdf', 'son hali'

Pull Request kuralları

- Her PR'da en az 1 comment — reviewer kodu inceleyip yazılı geri bildirim vermelidir
- PR açıklamasında ne yapıldığı kısaca yazılmalı
- Merge öncesi conflict varsa PR açan kişi çözecek

⚠ Commit geçmişini değerlendirmede incelenecektir. Yalnızca tek kişinin commit attığı repolar grup notu yerine bireysel not alır.

Bireysel Sorumluluk

Sunum grup sunumu olsa da değerlendirme bireyseldir. Her kişi aşağıdakileri karşılamalıdır:

Kendi kodun hakkında

- Geliştirdiğin her ekranı ve component'ı satır satır açıklayabilmelisin
- Neden o yaklaşımı seçtiğini gerekçelendirebilmelisin
- Context'e ne kattığını ve nasıl bağladığını gösterebilmelisin
- Sunumda anlık soru sorulabilir — hazırlıklı ol

Arkadaşlarının kodu hakkında

- Gruptaki diğer iki kişinin ekranlarını ve component'larını genel hatlarıyla anlamalısın
- Onların Context kullanımını açıklayabilmelisin
- PR review yaptığın kodda neyi neden onayladığını veya değişiklik istediğini söyleyebilmelisin
- 'O kısmı arkadaşım yaptı, bilmiyorum' geçerli bir cevap değildir

Değerlendirme Kriterleri

Kriter	Değerlendirilen	Ağırlık
Figma tasarımı	6 ekran tam çizilmiş, component sistemi kurulmuş, durumlar gösterilmiş	%15
Figma → Kod uyumu	Uygulama görünümü tasarımıyla birebir örtüşüyor mu?	%15
Context mimarisi	3 Context doğru kurulmuş, prop drilling yok, hook'lar ayrı	%20
Ekranlar arası senkron	6 senaryo tablosundaki akışlar doğru çalışıyor mu?	%20
Component yapısı	Klasör yapısı, yeniden kullanım, magic number yok	%10
GitHub disiplini	Her kişiden commit, anlamlı mesajlar, PR + review	%10
Bireysel bilgi	Kendi kodu + arkadaş kodunu açıklayabilme	%10

Teslim Edilecekler

- Figma linki (view only erişim açık, tüm ekranlar ve component'lar mevcut)
- GitHub repo linki (commit geçmişi, branch'ler ve PR'lar görünür olmalı)
- Çalışan demo — fiziksel cihaz veya Expo Go üzerinden
- README: kurulum adımları, test kullanıcısı, klasör yapısı açıklaması
- 7 dakika grup sunumu: Figma walkthrough → kod mimarisi → canlı demo → bireysel sorular

⚠ Sunum sırası: önce Figma gösterilecek, sonra uygulamanın aynı ekranları yan yana karşılaştırılacak. Ardından her kişiye kendi ekranlarından ve arkadaşının kodundan soru sorulacak.

Başlarken Önerilen Sıra

1. Figma'yı açın — önce renk, tipografi ve component sistemini kurun, sonra ekranları çizin
2. Figma tamamlandıncaya GitHub reposunu oluşturun, branch'lere bölünün
3. Yeni bir proje oluşturun, klasör yapısını kurun, navigation ekleyin
4. Üç Context'i boş olarak tanımlayın ve App.js'e provider'ları ekleyin
5. DummyJSON login'i test edin, token alındığını doğrulayın
6. Artık ekranlara paralel geliştirmeye geçilebilir — her kişi kendi branch'inde

⚠ Figma'yı atlamayın. 'Sonra yaparız' diyenler koda geçince tasarım kararlarıyla boğuşur ve iki katı zaman harcar.

Başarılar!